Task 1: Provision an Ubuntu EC2 Instance on AWS Using Terraform

1. Objective

To provision an Ubuntu 24.04 LTS EC2 instance on AWS using Terraform as Infrastructure as Code (IaC).

The EC2 instance will serve as the application host for the subsequent tasks involving Nginx, Docker Compose, reverse proxy configuration, logging, and log rotation.

2. Purpose

The purpose of this task is to understand how Terraform automates AWS infrastructure provisioning.

The following activities are performed:

- Configure Terraform with the AWS provider.
- Connect Terraform to the AWS account.
- Identify and select a valid Ubuntu AMI.
- Configure an EC2 Security Group.
- Provision an EC2 instance.
- Configure SSH access using an existing EC2 key pair.
- Verify the instance and connect to Ubuntu.

3. Environment and Infrastructure Details

Component	Configuration
Cloud Provider	AWS
Region	us-east-1
Operating System	Ubuntu 24.04.5 LTS
AMI ID	ami-0d6dd9e8265d48906
Instance Type	t3.micro
Architecture	x86_64
Key Pair	Devops
Private Key	devops.pem
Root Volume	8 GB gp3
SSH Port	22
Application Port	8080
Provisioning Tool	Terraform 1.17.0

4. Initial AWS Verification

Step 1: Verify AWS authentication

Command: aws sts get-caller-identity
The AWS CLI successfully returned the account identity.
AWS Account ID: 567487919764

Step 2: Configure and verify AWS Region

Command: aws configure get region
Output: us-east-1

Step 3: Verify the Ubuntu AMI

The originally supplied AMI was: ami-0461b1436c9b3c1a3

AWS returned: InvalidAMIID.NotFound

Therefore, a valid Ubuntu AMI was searched using Canonical's AWS account ID.

Command:

```
aws ec2 describe-images \
 --region us-east-1 \
 --owners 099720109477 \
 --filters \
  "Name=name,Values=ubuntu/images/hvm-ssd-gp3/ubuntu-noble-24.04-amd64-server-*" \
  "Name=state,Values=available" \
 --query 'sort_by(Images,&CreationDate)[-1].[ImageId,Name,CreationDate,Architecture]' \
 --output table
```

Verified AMI:
AMI ID: ami-0d6dd9e8265d48906
Ubuntu: 24.04 LTS
Architecture: x86_64

Step 4: Check existing EC2 key pairs

Command:

```
aws ec2 describe-key-pairs \
 --region us-east-1 \
 --query 'KeyPairs[*].[KeyName,KeyPairId,KeyType]' \
 --output table
```

Available key pairs included:

- Devops
- jenkins-key

The existing Devops key pair was selected.

The corresponding private key, devops.pem, was available in the Windows Downloads directory.

5. Terraform Project Structure

Created the following project directory:

```
cricbuzz-terraform/
|
 ├— provider.tf
 ├— main.tf
 ├— variables.tf
 ├— outputs.tf
```

```
├── terraform.tfvars
│
└── scripts/
```

Commands: created a directory

```
mkdir cricbuzz-terraform
cd cricbuzz-terraform
mkdir scripts
```

6. Terraform Configuration Files

6.1 provider.tf

This file defines the Terraform version requirement, AWS provider, and AWS Region.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

6.2 variables.tf

This file declares the input variables used throughout the Terraform configuration.

```
variable "aws_region" {
  description = "AWS region"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "Ubuntu 24.04 AMI ID"
  type        = string
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "key_name" {
```

```
  description = "Existing AWS EC2 key pair"
  type        = string
}

variable "ssh_cidr" {
  description = "Your public IP address in CIDR notation"
  type        = string
}
```

6.3 terraform.tfvars

This file supplies the actual values for the declared Terraform variables.

Final configuration:

```
aws_region    = "us-east-1"
ami_id        = "ami-0d6dd9e8265d48906"
instance_type = "t3.micro"
key_name      = "Devops"
ssh_cidr      = "124.123.159.234/32"
```

The SSH CIDR restricts SSH access to the configured public IP address.

The instance type was changed from t2.micro to t3.micro after AWS rejected the initial deployment because t2.micro was not eligible under the account's current Free Tier rules.

6.4 main.tf

This file defines the AWS infrastructure.

It includes:

1. Default VPC lookup.
2. Default VPC subnet lookup.
3. Security Group creation.
4. EC2 instance creation.
5. CPU credit configuration.
6. Root EBS volume configuration.

Complete configuration:

```
# Get the default VPC
data "aws_vpc" "default" {
  default = true
}

# Get subnets belonging to the default VPC
data "aws_subnets" "default" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.default.id]
  }
}

# Create Security Group
```

```
resource "aws_security_group" "cricbuzz_sg" {
  name        = "cricbuzz-terraform-sg"
  description = "Allow SSH and application traffic"
  vpc_id      = data.aws_vpc.default.id

  # SSH access
  ingress {
    description = "SSH access"
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = [var.ssh_cidr]
  }

  # Public web server access
  ingress {
    description = "Web server port 8080"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Outbound traffic
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "cricbuzz-security-group"
  }
}

# Create EC2 Instance
resource "aws_instance" "cricbuzz_server" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  key_name               = var.key_name
  subnet_id              = data.aws_subnets.default.ids[0]
  vpc_security_group_ids = [aws_security_group.cricbuzz_sg.id]
  associate_public_ip_address = true

  # T3 CPU credit configuration
  credit_specification {
    cpu_credits = "standard"
```

```
  }

  # Root EBS volume
  root_block_device {
    volume_size = 8
    volume_type = "gp3"
  }

  tags = {
    Name        = "Cricbuzz-Terraform-EC2"
    Environment = "Lab"
    ManagedBy   = "Terraform"
  }
}
```

6.5 outputs.tf

This file displays important EC2 information after Terraform provisions the infrastructure.

```
output "instance_id" {
  description = "EC2 instance ID"
  value       = aws_instance.cricbuzz_server.id
}

output "public_ip" {
  description = "Public IP address"
  value       = aws_instance.cricbuzz_server.public_ip
}

output "public_dns" {
  description = "Public DNS name"
  value       = aws_instance.cricbuzz_server.public_dns
}

output "security_group_id" {
  description = "Security Group ID"
  value       = aws_security_group.cricbuzz_sg.id
}
```

7. Terraform Execution Commands

Step 1: Initialize Terraform: terraform init

Step 2: Format Terraform files : terraform fmt -recursive

Purpose: Formats Terraform files according to standard HCL formatting.

Step 3: Validate the configuration: terraform validate

Step 4: Generate the execution plan: terraform plan

The plan showed: Plan: 2 to add, 0 to change, 0 to destroy.

Step 5: Apply the Terraform configuration: terraform apply

8. Final Infrastructure Verification

Step 1: Retrieve Terraform outputs: terraform output

- instance_id = "i-0f6dd4bae6cc77dd3"
- public_dns = "ec2-32-196-113-168.compute-1.amazonaws.com"
- public_ip = "32.196.113.168"
- security_group_id = "sg-09e1c197575215bb0"

Step 2: Connect to EC2 using SSH

On the Windows Git Bash terminal, the private key was available at: ~/Downloads/devops.pem

Set the private key permissions: chmod 400 ~/Downloads/devops.pem

Connect to Ubuntu: ssh -i ~/Downloads/devops.pem ubuntu@32.196.113.168

Successful connection displayed: ubuntu@ip-172-31-0-52:~$

Step 4: Verify the Ubuntu environment

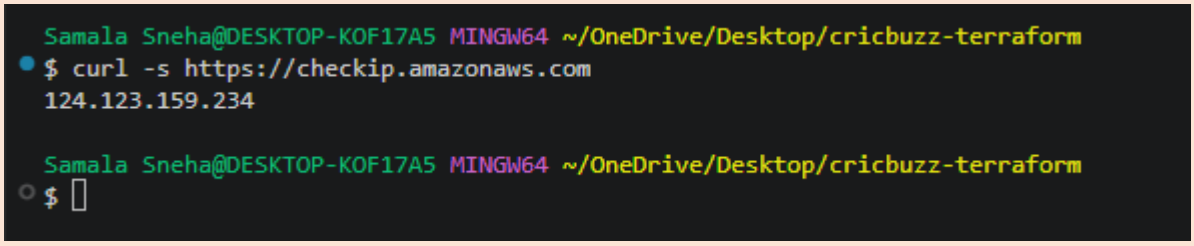
Commands:

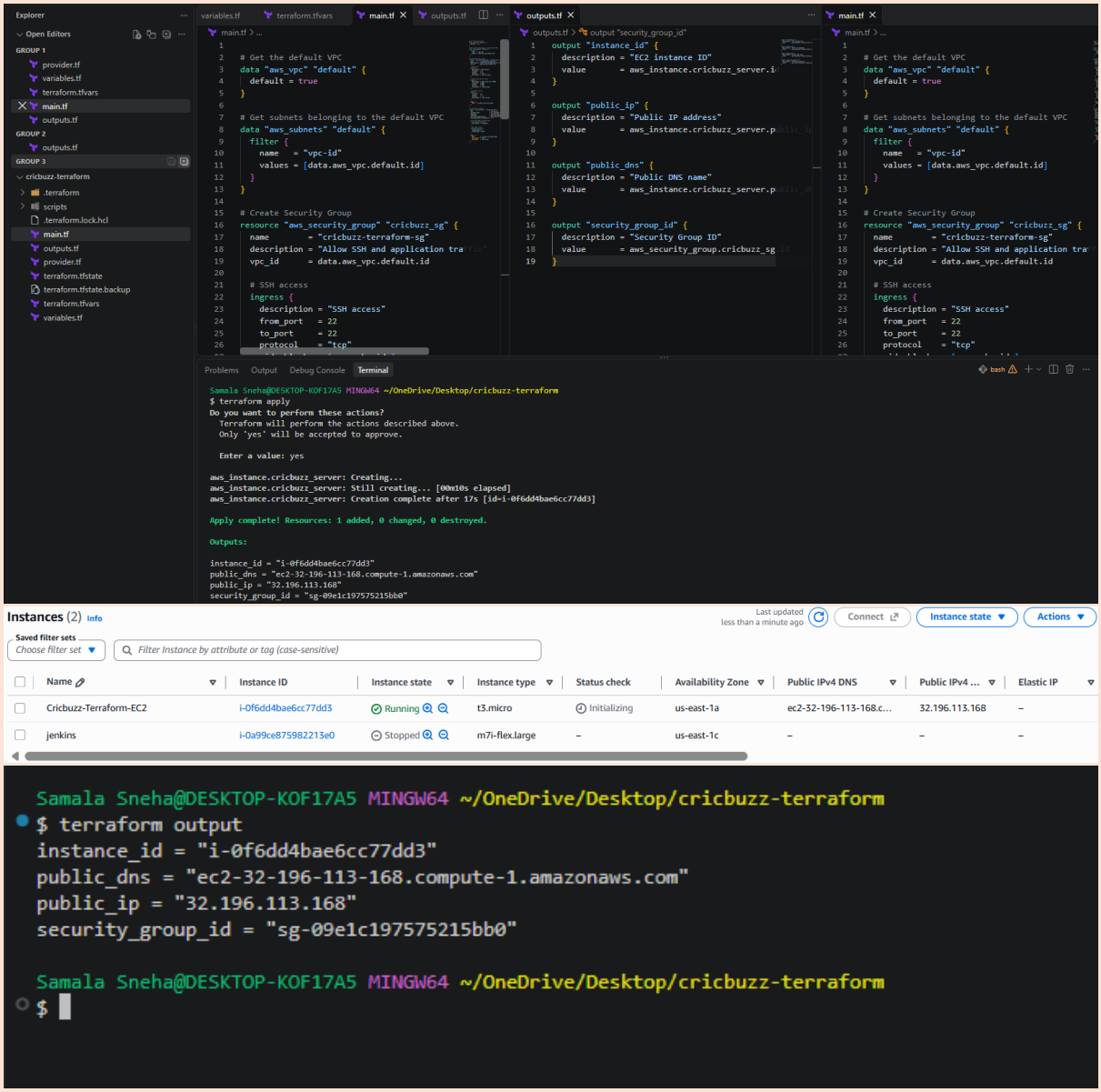
- Whoami
  hostname
  cat /etc/os-release

Actual results:

ubuntu
Hostname:
ip-172-31-0-52
Operating system:
PRETTY_NAME="Ubuntu 24.04.5 LTS"
VERSION_ID="24.04"
VERSION_CODENAME=noble
ID=ubuntu
This confirmed that the EC2 instance was successfully provisioned and accessible through SSH.

9. Implementation





10. Conclusion

Successfully provisioned an Ubuntu 24.04.5 LTS EC2 instance on AWS using Terraform.

The infrastructure was created using reusable Terraform configuration files, including provider configuration, input variables, resource definitions, and output values.

During deployment, an AWS Free Tier eligibility issue was encountered with t2.micro. This was investigated and resolved by selecting t3.micro after verifying its eligibility through the AWS EC2 API.

The EC2 instance was successfully created, verified in the AWS Console, and accessed using SSH.

Task 2 – Install and Configure Nginx on Port 8080 Using Terraform

1. Objective

Install and configure Nginx on the Ubuntu EC2 instance using Terraform. Configure Nginx to listen on port 8080 and serve a custom HTML page.

2. Purpose

The purpose of this task is to automate web server installation and configuration through Terraform.

Instead of manually connecting to the EC2 instance and installing Nginx, Terraform uses SSH provisioners to transfer and execute a shell script on the EC2 instance.

This establishes the web server that will later act as a reverse proxy for the Docker application.

4. Project Structure

The following files were used:

```
cricbuzz-terraform/
|
 ├── provider.tf
 ├── main.tf
 ├── variables.tf
 ├── outputs.tf
 ├── terraform.tfvars
 ├── nginx.tf
|
└── scripts/
   └── install_nginx.sh
```

5. Step-by-Step Execution

Step 1: Create the installation script

Created the directory and script: `mkdir -p scripts`

`touch scripts/install_nginx.sh`

The script automates the following operations:

- Update Ubuntu package repositories.
- Install Nginx and curl.
- Create the web root directory.
- Create a custom HTML page.
- Configure Nginx to listen on port 8080.
- Validate the Nginx configuration.
- Enable and restart the Nginx service.
- Verify the HTTP response.

File: scripts/install_nginx.sh

```
#!/bin/bash
set -eu

echo "Starting Nginx installation..."

apt-get update -y
apt-get install -y nginx curl

mkdir -p /var/www/html

cat > /var/www/html/index.html <<'HTML'
<!DOCTYPE html>
<html>
<head>
  <title>Cricbuzz Terraform Lab</title>
</head>
<body>
  <h1>Nginx is running successfully on port 8080</h1>
  <p>Deployed through Terraform.</p>
</body>
</html>
HTML

cat > /etc/nginx/sites-available/default <<'NGINX'
server {
  listen 8080 default_server;
  listen [::]:8080 default_server;

  server_name _;
  root /var/www/html;
  index index.html;

  location / {
    try_files $uri $uri/ =404;
  }
}
NGINX

nginx -t

systemctl enable nginx
systemctl restart nginx

systemctl is-active --quiet nginx

echo "Testing Nginx on port 8080..."
```

```
curl --fail http://127.0.0.1:8080/

echo "Nginx installation and HTTP verification completed successfully."
```

Step 2: Configure Terraform remote execution

Created **nginx.tf**.

```
resource "terraform_data" "nginx_setup" {

  triggers_replace = [
    aws_instance.cricbuzz_server.id,
    filesha256("${path.module}/scripts/install_nginx.sh")
  ]

  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.cricbuzz_server.public_ip
    private_key = file(pathexpand("~/Downloads/devops.pem"))
    timeout     = "5m"
  }

  provisioner "file" {
    source      = "${path.module}/scripts/install_nginx.sh"
    destination = "/tmp/install_nginx.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "chmod +x /tmp/install_nginx.sh",
      "sudo /bin/bash /tmp/install_nginx.sh"
    ]
  }
}
```

Explanation:

- terraform_data: Manages the execution of the provisioning workflow.
- triggers_replace: Re-executes the setup when the EC2 instance ID or installation script changes.
- connection: Establishes SSH connectivity using the existing EC2 key pair.
- file provisioner: Copies the installation script to the EC2 instance.
- remote-exec: Executes the script remotely.
- sudo /bin/bash: Explicitly invokes Bash to execute the script with administrative privileges.

This approach allows Terraform to rerun the configuration without unnecessarily replacing the EC2 instance.

Step 3: Format and validate Terraform

Executed these commands from the project directory:

- terraform fmt -recursive
- terraform validate

Expected validation result: Success! The configuration is valid.

Step 4: Review the execution plan: terraform plan

Step 5: Apply the configuration: terraform apply

6. Verification and Results

The following checks were included in the installation script:

Nginx configuration validation: nginx -t

Enable Nginx at system startup: systemctl enable nginx

Restart Nginx: systemctl restart nginx

Verify service status: systemctl is-active --quiet nginx

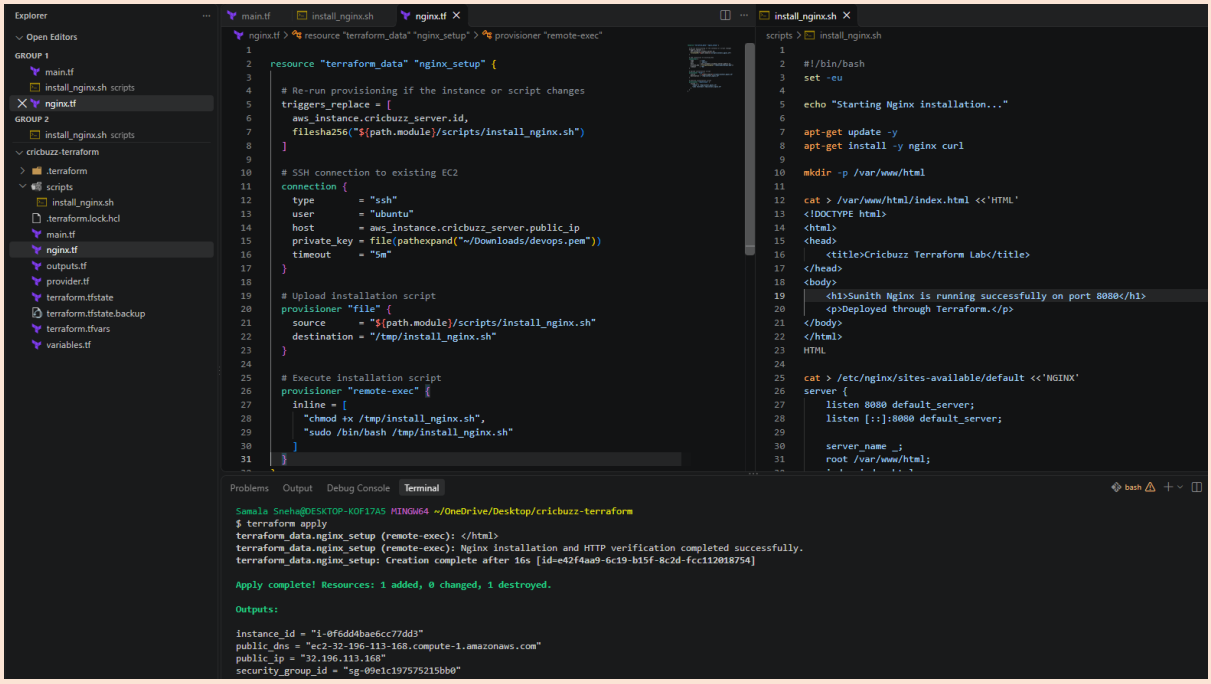
Verify the HTTP response locally: curl --fail http://127.0.0.1:8080/

The final page was also opened and checked in the browser using: http://32.196.113.168:8080

Observed result:

- Nginx installation completed.
- Nginx configuration was validated.
- Nginx was enabled and running.
- The custom HTML page was accessible through the browser on port 8080.
- The installation was performed through Terraform.

8.Implementation





9. Conclusion

Task 2 was successfully completed.

Nginx was installed and configured on the existing Ubuntu EC2 instance using Terraform remote provisioners. The server listens on port 8080 and serves a custom HTML page.

The configuration is automated through Terraform, and the browser test confirmed that the page is accessible.

This Nginx server will be used in the next stages to reverse-proxy traffic to the Docker Compose application running on port 8000.

Task 3: Install Docker, Configure Docker Compose, and Deploy the Cricbuzz Application Using Terraform

1. Objective

Install Docker Engine and Docker Compose on the existing Ubuntu EC2 instance, clone the Cricbuzz application repository, compile its Go application, build a Docker image, and deploy the application using Docker Compose.

The application must be accessible through host port 8000, which will later be used by Nginx as its reverse-proxy destination.

All server-side installation and configuration must be executed through Terraform.

2. Purpose

The purpose of this task is to automate application deployment using Terraform remote provisioners.

The deployment includes:

- Installing Docker Engine.
- Installing the Docker Compose plugin.
- Installing Go to compile the application.
- Cloning the GitHub repository.
- Generating the required application executable.
- Building a Docker image.
- Creating a Docker Compose configuration.

- Starting the application container.
- Verifying the container and HTTP response.

Repository: https://github.com/betawins/cricbuzz-docker-hello-world.git

EC2 instance: i-0f6dd4bae6cc77dd3

Application port mapping: 127.0.0.1:8000 → container:8080

3. Execution

Step 1: Create the installation script

File: scripts/install_docker_compose.sh

The script performs the complete installation and deployment through Terraform.

Step 2: Install prerequisites

```
apt-get update -y

apt-get install -y \
  ca-certificates \
  curl \
  git \
  golang-go
```

These packages provide the tools required to download Docker packages, clone the repository, and compile the Go application.

Step 3: Configure the official Docker repository

```
install -m 0755 -d /etc/apt/keyrings

curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc

chmod a+r /etc/apt/keyrings/docker.asc
```

Determine the Ubuntu release codename:

```
UBUNTU_CODENAME=$(
  . /etc/os-release
  echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}"
)
```

Add the Docker repository:

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu ${UBUNTU_CODENAME} stable" \
  > /etc/apt/sources.list.d/docker.list
```

Step 4: Install Docker Engine and Docker Compose

```
apt-get update -y

apt-get install -y \
  docker-ce \
  docker-ce-cli \
  containerd.io \
  docker-buildx-plugin \
  docker-compose-plugin
```

Enable and start Docker:

- systemctl enable --now docker
- systemctl is-active --quiet docker

Verify installed versions:

- docker --version
- docker compose version
- go version

Actual versions confirmed:

- Docker: 29.9.2
- Docker Compose: v5.6.1
- Go: 1.22.2

Step 5: Clone the Cricbuzz repository

Repository location: /opt/cricbuzz-docker-hello-world

```
APP_DIR="/opt/cricbuzz-docker-hello-world"
REPO_URL="https://github.com/betawins/cricbuzz-docker-hello-world.git"

if [ -d "$APP_DIR/.git" ]; then
  git -C "$APP_DIR" pull --ff-only
else
  mkdir -p /opt
  git clone "$REPO_URL" "$APP_DIR"
fi

cd "$APP_DIR"
```

Verify required source files:

```
for required_file in Dockerfile go.mod go.sum hello_world.go; do
  if [ ! -f "$required_file" ]; then
    echo "ERROR: Required repository file is missing: $required_file"
    exit 1
  fi
done
```

Step 6: Compile the Go application

The repository's Dockerfile expects an executable named hello_world.

The executable is generated from the Go source code before building the Docker image.

```
CGO_ENABLED=0 go build -a -o hello_world
```

Verify the executable:

```
if [ ! -s hello_world ] || [ ! -x hello_world ]; then
  echo "ERROR: Go executable was not generated."
  exit 1
fi
```

Terraform execution confirmed:

Go executable generated successfully.

Step 7: Configure the Docker build context

Create **.dockerignore**:

```
**
!Dockerfile
!hello_world
```

This excludes unnecessary repository files while retaining the Dockerfile and compiled application executable required by the image build.

Step 8: Create the Docker Compose configuration

File created on EC2: /opt/cricbuzz-docker-hello-world/compose.yaml

```
services:
 cricbuzz-app:
  build:
   context: .
   dockerfile: Dockerfile
  image: cricbuzz-app:local
  container_name: cricbuzz-app
  ports:
   - "127.0.0.1:8000:8080"
  restart: unless-stopped
```

Explanation:

- build.context: Uses the current application directory.
- dockerfile: Uses the repository's Dockerfile.
- image: Names the locally built image.
- container_name: Assigns the name cricbuzz-app.
- ports: Maps host port 8000 to container port 8080, bound only to localhost.
- restart: Restarts the container unless it is explicitly stopped.

Validate the Compose configuration: docker compose config --quiet

Step 9: Build and deploy the application: docker compose up -d --build

Step 10: Verify the container :docker compose ps

Verify the container's running state:

```
CONTAINER_RUNNING=$(
  docker inspect \
    --format='{{.State.Running}}' \
    cricbuzz-app
)

if [ "$CONTAINER_RUNNING" != "true" ]; then
  echo "ERROR: Application container is not running."
  docker compose logs --no-color
  exit 1
fi
```

Step 11: Verify the application HTTP response

```
SUCCESS=0

for attempt in $(seq 1 30); do
  if curl -fsS \
    http://127.0.0.1:8000/ \
    -o /tmp/cricbuzz-response.html; then
    SUCCESS=1
    break
  fi

  echo "Waiting for application... Attempt ${attempt}/30"
  sleep 2
done

if [ "$SUCCESS" -ne 1 ]; then
  echo "ERROR: Application HTTP verification failed."
  docker compose logs --no-color
  exit 1
fi

cat /tmp/cricbuzz-response.html
Actual HTTP response:
Hello, World!
```

4. Terraform Configuration

File:**docker.tf**

```
resource "terraform_data" "docker_setup" {
 triggers_replace = [
  aws_instance.cricbuzz_server.id,
  filesha256("${path.module}/scripts/install_docker_compose.sh")
 ]
```

```
  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.cricbuzz_server.public_ip
    private_key = file(pathexpand("~/Downloads/devops.pem"))
    timeout     = "10m"
  }

  provisioner "file" {
    source      = "${path.module}/scripts/install_docker_compose.sh"
    destination = "/tmp/install_docker_compose.sh"
  }

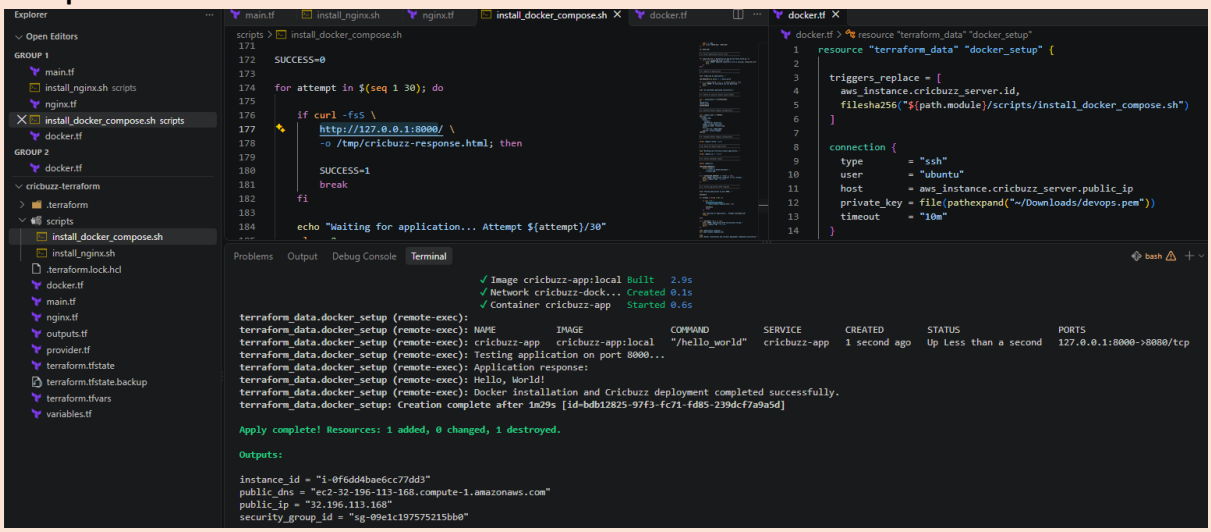
  provisioner "remote-exec" {
    inline = [
      "chmod +x /tmp/install_docker_compose.sh",
      "sudo /bin/bash /tmp/install_docker_compose.sh"
    ]
  }
}
```

5. Terraform Commands Used

- terraform fmt -recursive
- terraform validate
- terraform plan
- terraform apply

Review the plan before applying to ensure the existing EC2 instance is not scheduled for replacement.

6. Implementation



7. Conclusion

Task 3 was successfully completed through Terraform.

Docker Engine and Docker Compose were installed, the Cricbuzz GitHub repository was deployed, the Go application was compiled, and the Docker image was built successfully.

The cricbuzz-app container is running and responds with Hello, World! on host port 8000.

The application is intentionally bound to localhost on port 8000. In Task 4, Nginx will act as the reverse proxy and forward incoming requests from public port 8080 to this Docker application.

Task 4 – Configure Nginx as a Reverse Proxy for a Docker Application Using Terraform

1. Objective

Configure Nginx as a reverse proxy to forward incoming HTTP requests from port 8080 to the Docker application running internally on port 8000.

All configuration and deployment activities are performed through Terraform using provisioners.

2. Purpose

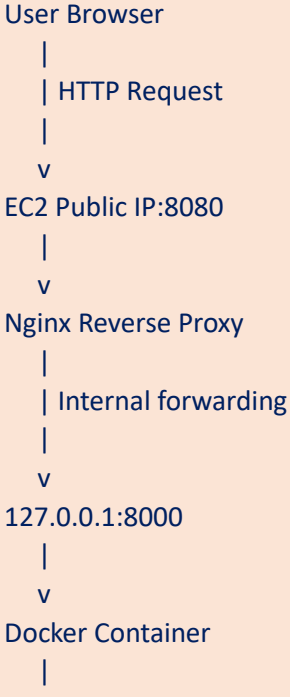
In Task 3, the Cricbuzz Docker application was successfully deployed on the EC2 instance and mapped to:

127.0.0.1:8000

However, the application was not directly accessible through the public IP because Docker was intentionally bound to the EC2 loopback interface.

The purpose of this task was to configure Nginx to act as a reverse proxy.

The resulting architecture:



v
Cricbuzz Application
|
v
Hello, World!

3. Implementation

Step 1: Navigate to the Terraform project

From Windows Git Bash: cd ~/OneDrive/Desktop/cricbuzz-terraform

Step 2: Create the Nginx proxy configuration script

Created: scripts/**configure_nginx_proxy.sh**

Script used:

```
#!/bin/bash
set -eu

echo "Starting Nginx reverse proxy configuration..."

# 1. Confirm Nginx configuration file exists
if [ ! -f /etc/nginx/sites-available/default ]; then
    echo "ERROR: Nginx default configuration not found."
    exit 1
fi

# 2. Configure Nginx reverse proxy
cat > /etc/nginx/sites-available/default <<'NGINX'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
NGINX

# 3. Validate Nginx configuration
echo "Validating Nginx configuration..."
```

```
nginx -t

# 4. Reload Nginx
systemctl reload nginx

# 5. Verify Nginx is active
systemctl is-active --quiet nginx

# 6. Verify Docker application is accessible through Nginx
echo "Testing reverse proxy..."

SUCCESS=0

for attempt in $(seq 1 30); do
  if curl -fsS http://127.0.0.1:8080/ \
    -o /tmp/cricbuzz-proxy-response.txt; then

    if grep -Fq "Hello, World!" /tmp/cricbuzz-proxy-response.txt; then
      SUCCESS=1
      break
    fi
  fi

  echo "Waiting for successful proxy response... Attempt ${attempt}/30"
  sleep 2
done

if [ "$SUCCESS" -ne 1 ]; then
  echo "ERROR: Reverse proxy verification failed."
  echo "Nginx response:"
  cat /tmp/cricbuzz-proxy-response.txt 2>/dev/null || true
  echo "Docker container status:"
  docker ps -a
  echo "Docker application logs:"
  docker logs cricbuzz-app 2>&1 || true
  exit 1
fi

echo "Reverse proxy response:"
cat /tmp/cricbuzz-proxy-response.txt

echo
echo "Nginx reverse proxy configured and verified successfully."
```

Step 3: Understand the Nginx configuration

The following configuration was written to: /etc/nginx/sites-available/default

```
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Explanation:

- listen 8080: Nginx accepts incoming HTTP requests on port 8080.
- listen [::]:8080: Enables listening on the IPv6 interface.
- server_name _: Acts as the default server name.
- location /: Matches requests to the root path and other paths.
- proxy_pass http://127.0.0.1:8000: Forwards requests to the Docker application.
- proxy_http_version 1.1: Uses HTTP/1.1 for communication with the upstream application.
- proxy_set_header Host $host: Preserves the original Host header.
- proxy_set_header X-Real-IP $remote_addr: Passes the original client IP.
- proxy_set_header X-Forwarded-For: Maintains the forwarded client IP information.
- proxy_set_header X-Forwarded-Proto: Passes the original request protocol.

Step 4: Create the Terraform configuration

Created:**nginx_proxy.tf**

Terraform configuration used:

```
resource "terraform_data" "nginx_proxy_setup" {

  triggers_replace = [
    aws_instance.cricbuzz_server.id,
    filesha256("${path.module}/scripts/configure_nginx_proxy.sh")
  ]

  depends_on = [
    terraform_data.nginx_setup,
    terraform_data.docker_setup
  ]
```

```
  connection {
    type        = "ssh"
    user        = "ubuntu"
    host        = aws_instance.cricbuzz_server.public_ip
    private_key = file(pathexpand("~/Downloads/devops.pem"))
    timeout     = "5m"
  }

  provisioner "file" {
    source      = "${path.module}/scripts/configure_nginx_proxy.sh"
    destination = "/tmp/configure_nginx_proxy.sh"
  }

  provisioner "remote-exec" {
    inline = [
      "chmod +x /tmp/configure_nginx_proxy.sh",
      "sudo /bin/bash /tmp/configure_nginx_proxy.sh"
    ]
  }
}
```

Step 5: Understand Terraform dependencies

The depends_on argument ensures that the Nginx proxy configuration is executed after the Nginx installation and Docker deployment resources.

Execution order:

1. EC2 instance creation.
2. Nginx installation.
3. Docker and Docker Compose installation and application deployment.
4. Nginx reverse proxy configuration.

The triggers_replace argument ensures that the Terraform data resource is replaced when the EC2 instance ID or proxy script changes.

Step 6: Format and validate Terraform

- terraform fmt
- terraform validate

Step 7: Review the execution plan: terraform plan

Step 8: Apply Terraform: terraform apply

Confirmed the execution when Terraform requested approval.

Terraform then:

1. Connected to the existing EC2 instance.
2. Uploaded the proxy configuration script.
3. Executed the script using Bash.
4. Validated the Nginx configuration using nginx -t.
5. Reloaded Nginx.
6. Checked the Nginx service.

7. Tested the reverse proxy using curl.
8. Verified that the application returned Hello, World!.

4. Verification and Results

A. Nginx configuration validation

The script executed: nginx -t

Expected successful result: nginx: configuration file /etc/nginx/nginx.conf test is successful

B. Nginx service verification

The script checked that Nginx was active: systemctl is-active --quiet nginx

C. Internal reverse proxy verification: curl -fsS http://127.0.0.1:8080/

It checked that the response contained:

Hello, World!

D. Public browser verification

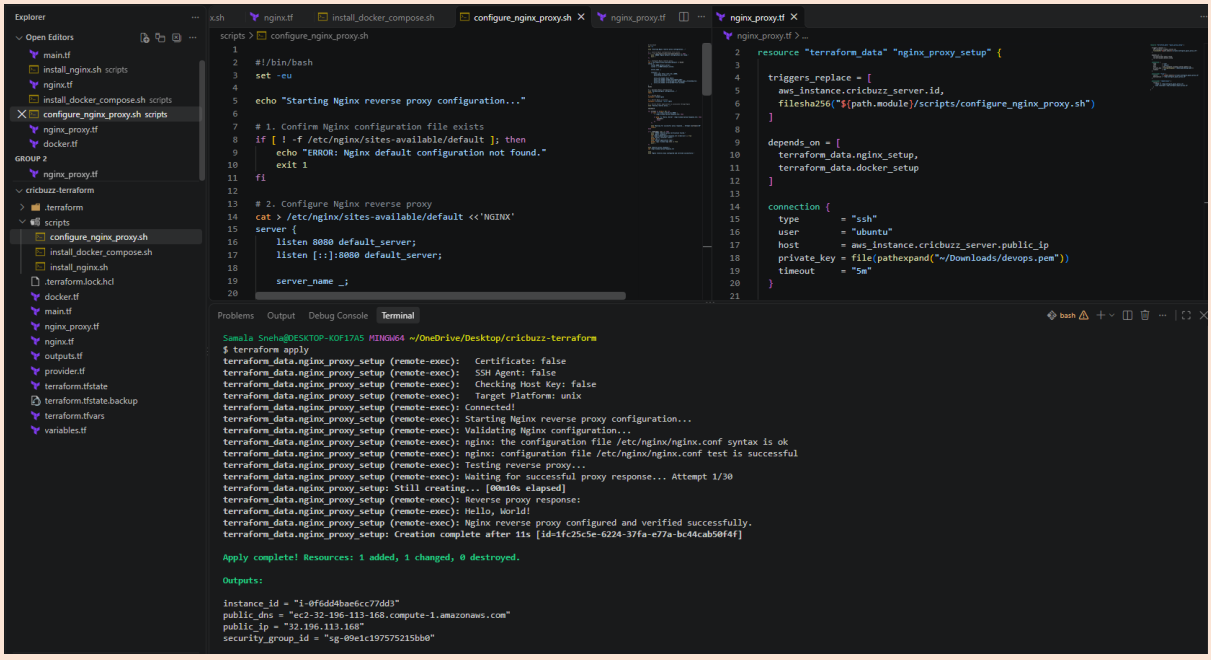
Opened the following URL from the Windows browser: http://32.196.113.168:8080/

Actual result: The application successfully displayed:

Hello, World!

The user confirmed that the application was accessible through the browser.

5. Implementation



```
root@ip-172-31-0-52:/home/ubuntu# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@ip-172-31-0-52:/home/ubuntu# systemctl is-active --quiet nginx
root@ip-172-31-0-52:/home/ubuntu# curl -fsS http://127.0.0.1:8080/
root@ip-172-31-0-52:/home/ubuntu# curl -fsS http://127.0.0.1:8080/
Hello, World!root@ip-172-31-0-52:/home/ubuntu#
```



6. Conclusion

Task 4 was successfully completed.

Nginx was configured as a reverse proxy to forward incoming requests on port 8080 to the Docker application running on port 8000.

The configuration was deployed through Terraform using terraform_data, the file provisioner, and the remote-exec provisioner.

Both internal HTTP verification and public browser verification were successful.

The Docker application remains bound to the EC2 loopback interface, so port 8000 does not need to be exposed publicly.

Task 5 – Configure Nginx Access Logging to /var/log/cb.log Using Terraform

1. Objective

Configure Nginx to record HTTP access logs in the file:

/var/log/cb.log

The configuration must preserve the existing Nginx reverse proxy and Docker application deployment.

All server-side configuration and execution must be performed through Terraform.

2. Purpose

In Task 4, Nginx was configured as a reverse proxy to forward incoming requests from port 8080 to the Docker application running on port 8000.

In this task, access logging was configured so that Nginx records incoming HTTP requests.

The access log contains useful information such as:

- Client IP address
- Request date and time
- HTTP request method and URL
- HTTP response status code
- Response size
- Referrer
- User-Agent

The required log file is: /var/log/cb.log

3. Implementation

Step 1: Navigate to the Terraform project

From Windows Git Bash: cd ~/OneDrive/Desktop/cricbuzz-terraform

Step 2: Create the Nginx logging script

Created: scripts/**configure_nginx_logging.sh**

The initial implementation was tested but failed to locate the generated access-log entry. The script was then improved with an explicit combined log format, a uniquely identifiable test request, and additional verification.

The final corrected script used successfully:

```
#!/bin/bash
set -eu

echo "Starting Nginx access logging configuration..."

LOG_FILE="/var/log/cb.log"
NGINX_CONFIG="/etc/nginx/sites-available/default"

# 1. Check prerequisites
```

```
if ! command -v nginx >/dev/null 2>&1; then
    echo "ERROR: Nginx is not installed."
    exit 1
fi

if [ ! -f "$NGINX_CONFIG" ]; then
    echo "ERROR: Nginx configuration file not found."
    exit 1
fi

# 2. Create the access log
touch "$LOG_FILE"
chown www-data:adm "$LOG_FILE"
chmod 0640 "$LOG_FILE"

# 3. Preserve reverse proxy and configure explicit access logging
cat > "$NGINX_CONFIG" <<'NGINX'
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    access_log /var/log/cb.log combined;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
NGINX

# 4. Validate configuration
echo "Validating Nginx configuration..."
nginx -t

# 5. Reload Nginx
systemctl reload nginx
systemctl is-active --quiet nginx

# 6. Generate a uniquely identifiable request
TEST_AGENT="Terraform-CB-Log-Verification"
TEST_URL="http://127.0.0.1:8080/?terraform_log_test=$(date +%s)"
```

```
echo "Sending test request through Nginx..."

curl --fail --silent --show-error \
  --user-agent "$TEST_AGENT" \
  "$TEST_URL" \
  -o /tmp/cb-log-test-response.txt

# 7. Verify application response
if ! grep -Fq "Hello, World!" /tmp/cb-log-test-response.txt; then
  echo "ERROR: Application response verification failed."
  cat /tmp/cb-log-test-response.txt
  exit 1
fi

# 8. Allow time for the access log entry and verify it
LOG_FOUND=0

for attempt in $(seq 1 10); do
  if grep -Fq "$TEST_AGENT" "$LOG_FILE"; then
    LOG_FOUND=1
    break
  fi

  sleep 1
done

if [ "$LOG_FOUND" -ne 1 ]; then
  echo "ERROR: Test request was not found in $LOG_FILE."

  echo "----- Access log file details -----"
  ls -l "$LOG_FILE"
  cat "$LOG_FILE" || true

  echo "----- Effective Nginx logging configuration -----"
  nginx -T 2>&1 | grep -n -E 'access_log|listen 8080|server_name' || true

  echo "----- Nginx service status -----"
  systemctl status nginx --no-pager || true

  echo "----- Nginx error log -----"
  tail -n 30 /var/log/nginx/error.log || true

  exit 1
fi

# 9. Display verification results
echo "Application response:"
```

```
cat /tmp/cb-log-test-response.txt

echo
echo "Verified access-log entry:"
grep -F "$TEST_AGENT" "$LOG_FILE"

echo
echo "Log file details:"
ls -l "$LOG_FILE"

echo
echo "Nginx access logging configured and verified successfully."
```

Step 3: Understand the Nginx access-log configuration

The script writes the following directive into: /etc/nginx/sites-available/default

access_log /var/log/cb.log combined;

Explanation:

- access_log: Enables access logging for the applicable Nginx context.
- /var/log/cb.log: Specifies the destination log file.
- combined: Uses Nginx's predefined combined log format.

The complete server configuration preserves the reverse proxy:

```
server {
    listen 8080 default_server;
    listen [::]:8080 default_server;

    server_name _;

    access_log /var/log/cb.log combined;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_http_version 1.1;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Step 4: Configure log file ownership and permissions The script executes:

- touch /var/log/cb.log
- chown www-data:adm /var/log/cb.log
- chmod 0640 /var/log/cb.log

Explanation:

- touch: Creates the log file if it does not already exist.
- chown www-data:adm: Assigns the file to the Nginx service account and the adm group.
- chmod 0640: Gives the owner read/write permissions, group read permission, and no permissions to others.

Step 5: Create the Terraform configuration

Created: **nginx_logging.tf**

Terraform configuration used:

```
resource "terraform_data" "nginx_logging_setup" {

 triggers_replace = [
  aws_instance.cricbuzz_server.id,
  filesha256("${path.module}/scripts/configure_nginx_logging.sh")
 ]

 depends_on = [
  terraform_data.nginx_proxy_setup
 ]

 connection {
  type       = "ssh"
  user       = "ubuntu"
  host       = aws_instance.cricbuzz_server.public_ip
  private_key = file(pathexpand("~/Downloads/devops.pem"))
  timeout    = "5m"
 }

 provisioner "file" {
  source      = "${path.module}/scripts/configure_nginx_logging.sh"
  destination = "/tmp/configure_nginx_logging.sh"
 }

 provisioner "remote-exec" {
  inline = [
   "chmod +x /tmp/configure_nginx_logging.sh",
   "sudo /bin/bash /tmp/configure_nginx_logging.sh"
  ]
 }
}
```

Step 6: Understand Terraform dependencies

The following dependency was configured:

```
depends_on = [
  terraform_data.nginx_proxy_setup
]
```

This ensures that access logging is configured after the Nginx reverse proxy setup.

The triggers_replace argument uses:

- EC2 instance ID
- SHA-256 hash of the logging script

This allows Terraform to recreate the provisioning resource when the instance or script changes.

Step 7: Format and validate Terraform

- terraform fmt
- terraform validate

Step 8: Apply Terraform: terraform apply

4. Verification and Results

A. Nginx configuration validation

The actual output:

- nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
- nginx: configuration file /etc/nginx/nginx.conf test is successful

B. Application response verification

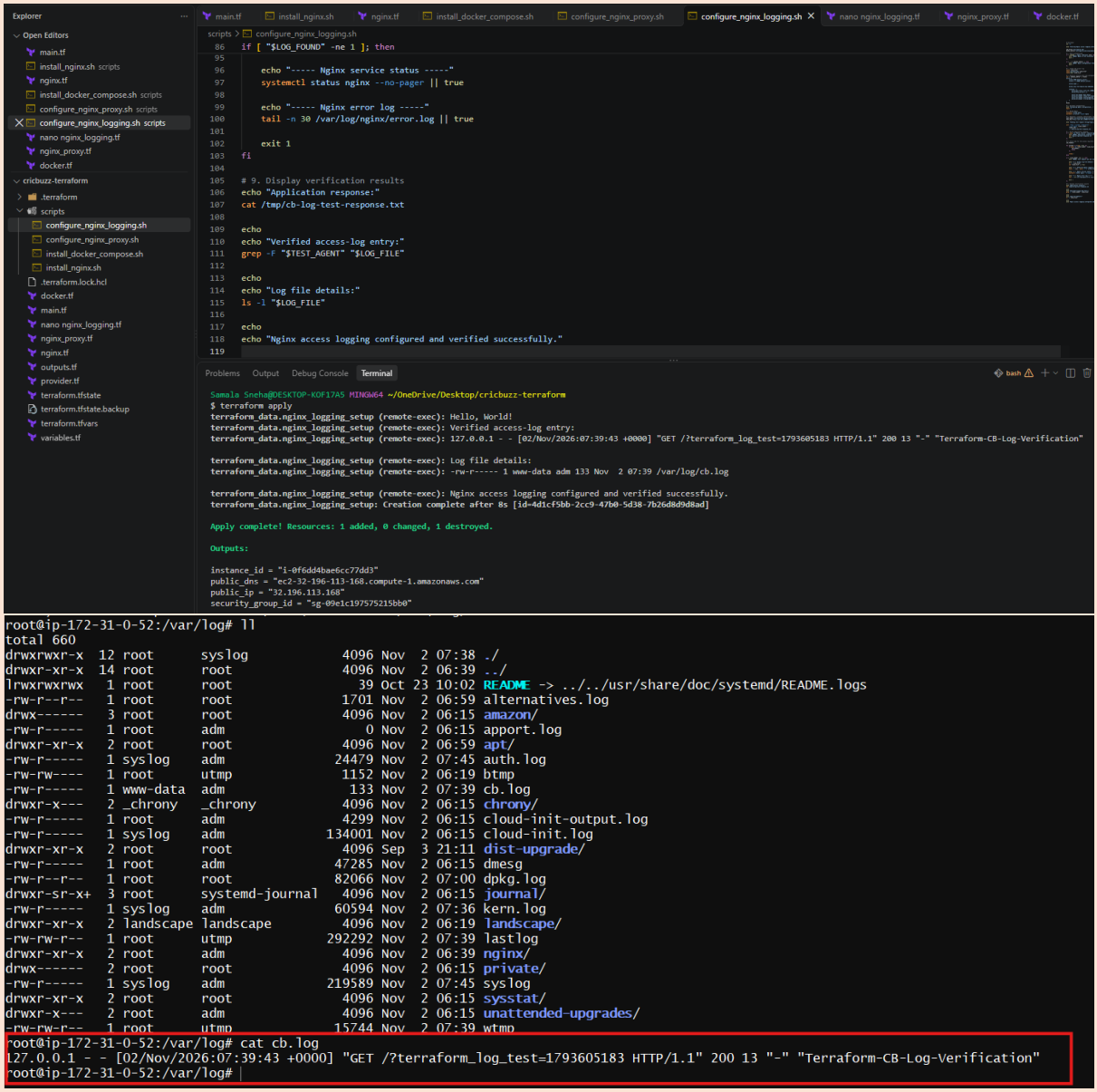
Actual output: Application response:

Hello, World!
This confirms that the reverse proxy continued to forward requests to the Docker application.

C. Access-log verification

Actual output:
Verified access-log entry:
127.0.0.1 - - [02/Nov/2026:07:39:43 +0000] "GET /?terraform_log_test=1793605183 HTTP/1.1" 200 13 "-" "Terraform-CB-Log-Verification"
This confirms that Nginx recorded the test HTTP request.

5. Implementation




```
root@ip-172-31-0-52:/var/log# ll
total 660
drwxrwxr-x  12 root      syslog            4096 Nov  2 07:38 ./
drwxr-xr-x  14 root      root              4096 Nov  2 06:39 ../
lrwxrwxrwx   1 root      root                39 Oct 23 10:02 README -> ../../usr/share/doc/systemd/README.logs
-rw-r--r--   1 root      root              1701 Nov  2 06:59 alternatives.log
drwx------   3 root      root              4096 Nov  2 06:15 amazon/
-rw-r-----   1 root      adm                  0 Nov  2 06:15 apport.log
drwxr-xr-x   2 root      root              4096 Nov  2 06:59 apt/
-rw-r-----   1 syslog    adm              24479 Nov  2 07:45 auth.log
-rw-rw----   1 root      utmp              1152 Nov  2 06:19 btmp
-rw-r-----   1 www-data  adm                133 Nov  2 07:39 cb.log
drwxr-x---   2 _chrony   _chrony           4096 Nov  2 06:15 chrony/
-rw-r-----   1 root      adm               4299 Nov  2 06:15 cloud-init-output.log
-rw-r-----   1 syslog    adm             134001 Nov  2 06:15 cloud-init.log
drwxr-xr-x   2 root      root              4096 Sep  3 21:11 dist-upgrade/
-rw-r-----   1 root      adm              47285 Nov  2 06:15 dmesg
-rw-r--r--   1 root      root             82066 Nov  2 07:00 dpkg.log
drwxr-sr-x+  3 root      systemd-journal   4096 Nov  2 06:15 journal/
-rw-r-----   1 syslog    adm              60594 Nov  2 07:36 kern.log
drwxr-xr-x   2 landscape landscape         4096 Nov  2 06:19 landscape/
-rw-rw-r--   1 root      utmp            292292 Nov  2 07:39 lastlog
drwxr-xr-x   2 root      adm              4096 Nov  2 06:39 nginx/
drwx------   2 root      root             4096 Nov  2 06:15 private/
-rw-r-----   1 syslog    adm             219589 Nov  2 07:45 syslog
drwxr-xr-x   2 root      root             4096 Nov  2 06:15 sysstat/
drwxr-x---   2 root      adm              4096 Nov  2 06:15 unattended-upgrades/
-rw-rw-r--   1 root      utmp            15744 Nov  2 07:39 wtmp
root@ip-172-31-0-52:/var/log# cat cb.log
127.0.0.1 - - [02/Nov/2026:07:39:43 +0000] "GET /?terraform_log_test=1793605183 HTTP/1.1" 200 13 "-" "Terraform-CB-Log-Verification"
root@ip-172-31-0-52:/var/log#
```

6. Conclusion

Task 5 was successfully completed.

Nginx access logging was configured through Terraform, and HTTP requests are now recorded in /var/log/cb.log.

The reverse proxy and Docker application continued working after the logging configuration was applied.

The log file's existence, ownership, permissions, and actual recorded HTTP request were verified through the Terraform provisioner output.

Task 6: Configure Log Rotation for Nginx Access Logs Using Terraform

1. Objective

Configure logrotate on the Ubuntu EC2 instance to automatically manage the Nginx access log stored at:

/var/log/cb.log

The configuration should rotate logs daily, retain seven rotated files, compress older logs, and recreate the active log with the correct ownership and permissions.

All server-side configuration is executed through Terraform using a terraform_data resource and provisioners.

2. Purpose

Nginx continuously generates access logs whenever clients access the application.

Without log rotation, the access log can grow indefinitely and consume disk space.

Logrotate helps manage log files by:

- Rotating logs periodically.
- Retaining a defined number of old log files.
- Compressing older logs to save storage.
- Creating a new active log file.
- Maintaining the required file ownership and permissions.
- Reopening Nginx log files after rotation.

3. Existing Environment

Component	Configuration
Operating system	Ubuntu 24.04.5 LTS
AWS region	us-east-1
EC2 instance type	t3.micro
Nginx listening port	8080
Docker application port	8000 (localhost only)
Nginx access log	/var/log/cb.log
Log owner	www-data
Log group	adm
Log permissions	0640
Logrotate version	3.21.0

4. Terraform Project Structure

The relevant project files are:

```
cricbuzz-terraform/
|
├── provider.tf
├── main.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars
|
├── nginx.tf
├── docker.tf
├── nginx_proxy.tf
├── nginx_logging.tf
├── logrotate.tf
|
└── scripts/
    ├── install_nginx.sh
    ├── install_docker_compose.sh
    ├── configure_nginx_proxy.sh
    ├── configure_nginx_logging.sh
    └── configure_logrotate.sh
```

5. Step-by-Step Execution

Step 1: Create the logrotate installation and configuration script

Created: scripts/**configure_logrotate.sh**

The script performs the following operations:

1. Installs logrotate.
2. Checks that Nginx and the existing access log are available.
3. Confirms that Nginx is configured to write to /var/log/cb.log.
4. Sets the correct ownership and permissions.
5. Creates the logrotate configuration.
6. Validates the configuration.
7. Generates an HTTP request before rotation.
8. Forces a controlled rotation test.
9. Verifies the rotated log archive.
10. Verifies the newly created active log.
11. Reopens Nginx log files.
12. Sends another HTTP request and verifies that Nginx writes to the new log.

Step 2: Configure the logrotate policy

The script creates: /etc/logrotate.d/cb

Configuration:

```
/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty

    su root adm

    create 0640 www-data adm

    sharedscripts
    postrotate
        nginx -s reopen
    endscript
}
```

Explanation:

Directive	Purpose
/var/log/cb.log	Specifies the log file to rotate.
daily	Requests daily rotation.
rotate 7	Retains seven rotated log files.
compress	Compresses older rotated logs.
delaycompress	Delays compression until the next rotation.
missingok	Does not report an error if the log file is missing.
notifempty	Does not rotate an empty log file.
su root adm	Specifies the user and group used for rotation.
create 0640 www-data adm	Creates the new log with the required permissions and ownership.
sharedscripts	Runs the post-rotation script once for the log group.
postrotate	Defines the command executed after rotation.
nginx -s reopen	Requests Nginx to reopen its log files.

Step 3: Understand the su directive

During the first execution, logrotate reported:

error: skipping "/var/log/cb.log" because parent directory has insecure permissions

Set "su" directive in config file

The issue was caused by logrotate's security check on the parent directory.

The configuration was corrected by adding: su root adm

This specifies the user and group under which log rotation should take place.

The permissions of /var/log were not weakened or changed.

Step 4: Configure Terraform

Created: **logrotate.tf**

```
resource "terraform_data" "logrotate_setup" {
 triggers_replace = [
  aws_instance.cricbuzz_server.id,
  filesha256("${path.module}/scripts/configure_logrotate.sh")
 ]

 depends_on = [
  terraform_data.nginx_logging_setup
 ]

 connection {
  type        = "ssh"
  user        = "ubuntu"
  host        = aws_instance.cricbuzz_server.public_ip
  private_key = file(pathexpand("~/Downloads/devops.pem"))
  timeout     = "10m"
 }

 provisioner "file" {
  source      = "${path.module}/scripts/configure_logrotate.sh"
  destination = "/tmp/configure_logrotate.sh"
 }

 provisioner "remote-exec" {
  inline = [
   "chmod +x /tmp/configure_logrotate.sh",
   "sudo /bin/bash /tmp/configure_logrotate.sh"
  ]
 }
}
```

Step 5: Understand the Terraform configuration

terraform_data Used to execute the configuration script through Terraform.

triggers_replace

```
triggers_replace = [
  aws_instance.cricbuzz_server.id,
  filesha256("${path.module}/scripts/configure_logrotate.sh")
]
```

The resource is replaced when the EC2 instance ID or script content changes.

depends_on

```
depends_on = [
  terraform_data.nginx_logging_setup
]
```

Ensures that Nginx access logging is configured before logrotate setup begins.

file provisioner Transfers the local script to the EC2 instance.

remote-exec provisioner Executes the uploaded script using Bash with elevated privileges.

Step 6: Execute Terraform

- terraform fmt
- terraform validate
- terraform plan
- terraform apply

6. Verification and Results

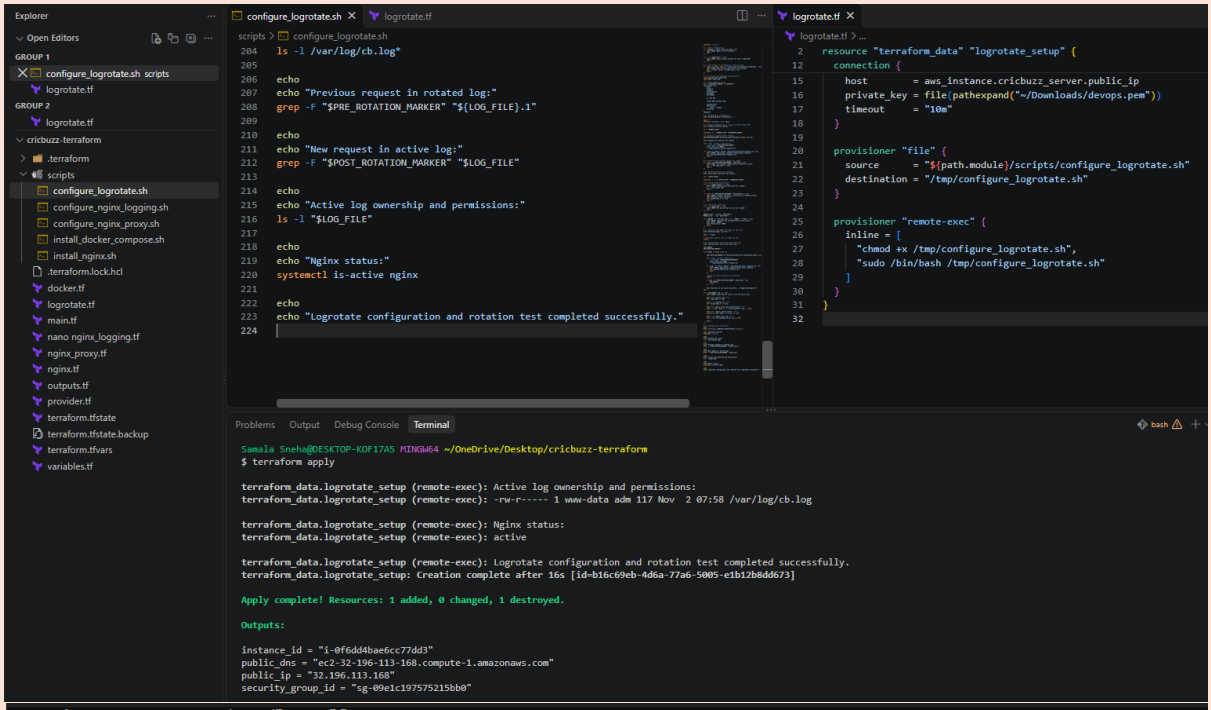
The final script was designed to verify the following:

Verification	Expected result
Logrotate installation	Version 3.21.0
Logrotate policy	Valid configuration
Pre-rotation HTTP request	Application responds successfully
Pre-rotation log entry	Present in the original log
Controlled rotation	Rotated archive created
Previous log entry	Present in rotated archive
New active log	Created successfully
New log ownership	www-data:adm
New log permissions	0640
Nginx log reopening	Requested successfully
Post-rotation HTTP request	Application responds successfully
New active log entry	Present in the new log

The user confirmed that the final terraform apply executed successfully.

Note: The complete final apply output was not included in the confirmation message, so individual verification lines are not reproduced as observed output here.

8. Implementation




```
root@ip-172-31-0-52:/var/log# ll
total 692
drwxrwxr-x  12 root      syslog          4096 Nov  2 07:57 ./
drwxr-xr-x  14 root      root            4096 Nov  2 06:39 ../
lrwxrwxrwx   1 root      root              39 Oct 23 10:02 README -> ../../usr/share/doc/systemd/README.logs
-rw-r--r--   1 root      root            1701 Nov  2 06:59 alternatives.log
drwx------   3 root      root            4096 Nov  2 06:15 amazon/
-rw-r-----   1 root      adm                0 Nov  2 06:15 apport.log
drwxr-xr-x   2 root      root            4096 Nov  2 06:59 apt/
-rw-r-----   1 syslog    adm            30418 Nov  2 07:58 auth.log
-rw-rw----   1 root      utmp            1152 Nov  2 06:19 btmp
-rw-r-----   1 www-data  adm              117 Nov  2 07:58 cb.log
-rw-r-----   1 www-data  adm              116 Nov  2 07:57 cb.log.1
-rw-r-----   1 www-data  adm              184 Nov  2 07:55 cb.log.2.gz
```

9. Conclusion

Successfully executed the logrotate configuration through Terraform.

The solution defines a daily rotation policy for the Nginx access log, retains seven rotated files, supports compression, recreates the active log with the required ownership and permissions, and instructs Nginx to reopen its log files after rotation.

The complete six-task Cricbuzz deployment assignment has now been executed using Terraform.